

THE INDEPENDENT INSTITUTE OF EDUCATION | IIE ROSEBANK COLLEGE

iTHRIFT CLOTHES

South African pre-loved branded fashion: website and Android application

XISD6329 Work Integrated Learning 3B

Task 2: Mobile application development, website update and DevOps implementation

Item	Detail
Module	XISD6329 Work Integrated Learning 3B
Assessment	Task 2: Working prototype (50% of the module)
Group	11 (Eleven)
Group members	Vukosi Rikhotso (ST10439408) Theo Golele (ST10439863)
Qualification	Diploma in Information Technology in Software Development
Institution	IIE Rosebank College, Pretoria
Client	iTHRIFT Clothes, Pretoria
Repository	https://github.com/ST10439408-VRIKHOTSO/iTHRIFT
Application version	1.1.0 (build 2)
Date	September 2026

Table of Contents

1. Introduction.....	5
1.1 Purpose of this document.....	5
1.2 The solution, in brief.....	5
1.3 Project links.....	5
1.4 How the prototype was developed.....	6
1.5 Acting on the Task 1 feedback.....	6
2. Team accountability and collaboration.....	7
2.1 Deliverables and ownership.....	7
2.2 How accountability was enforced.....	7
3. DevOps implementation.....	8
3.1 Planning on Azure Boards.....	8
3.2 Source control and branching.....	8
3.3 Continuous integration with GitHub Actions.....	9
3.4 Release and deployment.....	10
3.5 Monitoring and logging.....	10
4. The Android application.....	11
4.1 Architecture.....	11
4.2 Program modules and project structure.....	12
4.3 The graphical user interface.....	12
Dark theme.....	12
4.4 Input controls and validation.....	13
4.5 Settings.....	14
5. Authentication, roles and security.....	14
5.1 Registration and sign-in.....	14
5.2 Password encryption.....	15
5.3 Single sign-on.....	16
Configuration, and why the button can be hidden.....	17
5.4 Roles and server-side access control.....	17
6. Database design.....	18
6.1 Database management system.....	18
6.2 Tables and relationships.....	18
6.3 Recording how an account signs in.....	18
6.4 Seed data.....	19
7. The shared REST API.....	21
8. Website updates.....	22
9. Automated testing.....	25
9.1 Kotlin unit tests.....	25
9.2 API unit tests.....	26
9.3 End-to-end smoke test.....	27

9.4 Results.....	27
10. Verification and validation.....	29
11. Help system and user documentation.....	30
12. Demonstration video.....	30
12.1 What the walk-through covers.....	30
13. Release notes.....	31
14. Preparation for Google Play publication.....	32
14.1 Draft store listing.....	33
15. Use of AI tools.....	33
16. Requirement checklist.....	33
16.1 Against the marking rubric.....	34
16.2 Against the module manual, Section 9.2.....	34
References.....	35

List of Figures

Figure 1: Rows in every table after npm run init-db; each meets the ten-record minimum.....	20
Figure 2: The Customer table: passwords are held only as a per-account salt and an scrypt hash.....	20
Figure 3: The Product table, first fifteen of 63 rows, with brand and category as foreign keys.....	21
Figure 4: Orders joined to Payment: twelve orders across all four statuses and three payment methods.....	21
Figure 5: Website, Shop: the catalogue with category filters and condition grades.....	23
Figure 6: Website, Product detail: size, condition, stock and reviews.....	23
Figure 7: Website, My Orders: order tracking with the courier reference.....	24
Figure 8: Website, Admin console: dashboard with revenue by brand and category.....	24
Figure 9: Website, Admin console: the order queue that staff work through.....	25
Figure 10: API unit tests: 9 passed, 0 failed (25 September 2026).....	28
Figure 11: End-to-end smoke test: 70 passed, 0 failed (25 September 2026).....	28
Figure 12: The API request log written during that run, showing refused requests logged as failures.....	29

1. Introduction

1.1 Purpose of this document

Task 1 produced the updated project plan, the site maps and the wireframes for the iTHRIFT Clothes solution. Task 2 is where that plan became a working product. This document records what Group 11 built, how it was tested, and where each requirement in Section 9.2 of the module manual has been met. It is the written companion to the code in the repository, to the pipeline that builds it, and to the demonstration video.

The scope of Task 2 is the working prototype: a native Android application that runs on a physical handset, the updated website, the shared REST API and relational database that both clients use, and the DevOps lifecycle that builds, tests and releases them. The final report, the peer and self evaluations and the presentation belong to Task 3 and are not repeated here.

Everything described in this document is in the repository and can be run. Where a claim is better shown by a screenshot from the running system than by a description, an amber box marks it.

1.2 The solution, in brief

iTHRIFT Clothes buys, grades and resells pre-loved branded clothing in South Africa. Before this project the business ran on WhatsApp and Instagram, with stock held in one person's memory and orders tracked in chat threads. The solution replaces that with two connected sales channels and one administrative console:

- a native Android application for customers, who browse, search, buy and track orders on a phone;
- an updated website carrying both the customer storefront and the staff and administrator console;
- one REST API and one relational database that both clients share, so a business rule is written once and enforced everywhere.

The architecture is the three-tier design carried forward from Task 1. The presentation tier now has two occupants, the application tier is a JSON REST API, and the data tier is a Third Normal Form relational schema. Neither client is trusted: cart totals, stock locking at checkout, order status transitions and role permissions are all enforced on the server. Hiding a control in a client is a convenience, never a control.

The shape of the system



1.3 Project links

These give the marker direct access to the repository, the pipeline and the running evidence. They are gathered here and repeated in the sections they belong to.

Resource	Link
GitHub repository	https://github.com/ST10439408-VRIKHOTSO/iTHRIFT
GitHub Actions (all pipeline runs)	https://github.com/ST10439408-VRIKHOTSO/iTHRIFT/actions

Resource	Link
README (setup, demo accounts, release notes)	https://github.com/ST10439408-VRIKHOTSO/iTHRIFT/blob/main/README.md
User guide	https://github.com/ST10439408-VRIKHOTSO/iTHRIFT/blob/main/docs/user-guide.md
Azure DevOps board	PASTE YOUR AZURE BOARDS URL HERE
Demonstration video (unlisted YouTube)	PASTE YOUR UNLISTED YOUTUBE LINK HERE

Table 1: Where to find each part of the submission

BEFORE YOU SUBMIT

Two links above are still placeholders: the Azure Boards URL and the demonstration video. Paste them in, and put the video link in the README as well.

Open every link in a private browser window first. The lecturer must be able to reach the repository, the board, the Actions history and the video without signing in as you.

1.4 How the prototype was developed

Development followed the pacer in the module manual and the guidance given in the weekly collaboration sessions. Implementation began once the Task 1 design was signed off, with the lecturer confirming the two-channel scope and the API-first sequence. Building the API first was the decision that shaped everything after it: with the contract fixed, the website and the application could be worked on in parallel without either developer waiting on the other.

The team met twice a week: one session with the lecturer to raise hurdles and confirm direction, and one internal stand-up to review the board and re-balance work. Any change to the agreed API contract was recorded as a work item, so that both developers were always building against the same agreement.

1.5 Acting on the Task 1 feedback

Task 1 scored 91 out of 100. The lecturer's marks showed where the remaining nine were lost, and each of those points was addressed before any Task 2 evidence was captured, so the screenshots in this document show the corrected system. The table sets out what was changed for each criterion.

Task 1 criterion	Mark	What we changed for Task 2
Cover page, contents, list of figures	9 / 10	This document now carries a list of figures as well as a table of contents, and every screenshot is numbered and captioned so the marker can find it from either list.
Update project plan	9 / 10	The plan is carried forward rather than restated. Section 3 reports what each DevOps phase actually looked like in practice, and Section 2.1 records who owned each deliverable.
File and folder structure	8 / 10	Renamed the 37 product images that used underscores (adidas_eqt.jpg is now adidas-eqt.jpg) and updated the seed script to match. Renamed both task documents to lowercase, hyphenated names with no spaces or commas, for example xisd6329-task1-group11-project-plan.docx. New evidence images follow the same rule.
HTML page code	17 / 20	Validated the static pages and the rendered output of every website route with html-validate, using its recommended and WCAG rule sets. Fixed the 38 table header cells that had no scope attribute and the 6

Task 1 criterion	Mark	What we changed for Task 2
		buttons with no type attribute, and pinned the icon library to a fixed version instead of @latest, so a third-party release can no longer change the site without us. Every page now validates with no errors.
Web page content	13 / 15	Edited the storefront copy so that sentences are no longer joined with dashes, which read as unfinished on a small screen. Eight phrases were rewritten, including the catalogue heading, the product button and the checkout note. Emoji icons were also removed from the navigation, buttons and form labels on both the website and the mobile web app, so every label is plain text.
Client needs, site map, wireframes, GitHub	10, 10, 10, 5	Full marks, so these were kept. The Android screens and the updated website follow the Task 1 wireframes.

Table 2: How each point of the Task 1 feedback was acted on

The file-naming point deserved the most attention, because it lost the most marks and because it is the easiest to regress on: a single photograph saved from a phone with a default name brings it back. Renaming the images meant changing the seed script in the same commit, since the database stores each product's image file name, and the end-to-end suite was run again afterwards to confirm that every product still loads its photograph.

2. Team accountability and collaboration

Task 2 is a group deliverable and each member is accountable for the items allocated to them. Ownership here means the named member is responsible for finishing the deliverable and making it work; the supporting member can still contribute. The allocation carries over from Task 1 and reflects who actually built what.

2.1 Deliverables and ownership

Deliverable	Owner	Support
REST API endpoints and business rules	Vukosi Rikhotso	Theo Golele
Database schema, seed data and constraints	Vukosi Rikhotso	None
Authentication, roles and single sign-on	Vukosi Rikhotso	Theo Golele
Android UI: Cart, Checkout, Orders, Account	Vukosi Rikhotso	Theo Golele
Android UI: Shop, Search, Product detail	Theo Golele	Vukosi Rikhotso
Settings and Help screens	Vukosi Rikhotso	Theo Golele
Website markup, styling and client logic	Theo Golele	Vukosi Rikhotso
Unit tests and GitHub Actions workflows	Vukosi Rikhotso	Theo Golele
Help system and user documentation	Theo Golele	Vukosi Rikhotso
Demonstration video and release notes	Theo Golele	Vukosi Rikhotso
Signed release build and Play Store assets	Vukosi Rikhotso	Theo Golele

Table 3: Deliverables and who owned each one

2.2 How accountability was enforced

Accountability rested on the board rather than on trust. Every work item had a single assignee and an acceptance test written before the work started, so "done" was not a matter of opinion. Any item that had not moved for a week was raised at the weekly session and either unblocked or reassigned.

Because the group has only two members, every deliverable also names a supporting member who can take it over. That is the mitigation for the highest-rated risk in the Task 1 risk register: in a group this small, one person being unavailable for a fortnight is not a delay, it is a failure, unless somebody else already knows the code.

EVIDENCE TO INSERT

A short meeting log giving dates, who attended and what was decided, or a screenshot of the board's activity feed, so the marker can see that regular meetings took place.

3. DevOps implementation

The module asks for the full DevOps lifecycle rather than a single hand-over at the end. The team used Azure Boards to plan, GitHub to hold the code, and GitHub Actions to build and test every change automatically. The eight phases below are the ones set out in the Task 1 plan; this section is what each one actually looked like in practice.

Phase	Tool	What happens
Plan	Azure Boards (Kanban)	Work packages become epics, then day-sized items, each with an assignee and an acceptance test.
Code	GitHub, VS Code, Android Studio	One repository holds the website, the API and the app. Work happens on short-lived feature branches.
Build	GitHub Actions, Gradle, npm	Two workflows run on every push: one assembles the Android debug build, one installs and starts the API.
Test	JUnit, node:test, smoke test	128 automated checks. A red run blocks the merge.
Release	Tag-triggered workflow	A v* tag produces a signed APK and App Bundle plus the release notes for that version.
Deploy	Play Console / host	The APK is installed on a physical handset for the demonstration and prepared for the Play Store.
Operate	Seeded demo environment	A seeded database and known demo accounts keep the walk-through repeatable.
Monitor	Server logging, Logcat	Per-request logging on the API and network-boundary logging in the app; anything found becomes a work item.

Table 4: The eight DevOps phases as applied to the iTHRIFT prototype

3.1 Planning on Azure Boards

The backlog was loaded onto a Kanban board with the columns New, Active, Review, Done and Later. The Later column is where interesting but unassessed ideas go; it is the team's guard against scope creep, and it works because an idea parked somewhere visible stops being argued about. Each item links to the branch that implements it, so the board and the code cannot drift apart unnoticed.

EVIDENCE TO INSERT

A screenshot of the Azure Boards Kanban board with items across the columns and the assignees visible.

3.2 Source control and branching

All three products live in one GitHub repository, which is why a change to a shared contract, such as an API route or a JSON field name, can be made in a single commit rather than in three that have to land together. The team works on short-lived feature branches named after the work item, opens a pull request, and merges to main only once both build workflows are green. The repository was initialised with a README and a .gitignore, and code is committed and pushed as it is written rather than in one large drop at the end. In line with the module rule, nothing is distributed as a ZIP file.

Branching model and a typical change

```
main                                protected; merges only via reviewed pull request
  feature/6.4-settings-screen
  feature/5.3-auth-sso
  feature/7.1-kotlin-unit-tests

# a typical change
git checkout -b feature/6.4-settings-screen
git add . && git commit -m "6.4 settings screen: profile, theme, connection"
git push -u origin feature/6.4-settings-screen
# open PR -> both workflows run -> review -> squash-merge to main
```

3.3 Continuous integration with GitHub Actions

Three workflows live in .github/workflows/. Two of them run on every push and every pull request to main; the third runs only on a release tag, because releasing is a decision and not a side effect of committing.

Workflow	Trigger	What it does
android.yml	push, PR	Sets up JDK 17, runs the 49 Kotlin unit tests, assembles the debug build, uploads the APK as an artefact.
api-and-website.yml	push, PR	Sets up Node 24, runs the 9 API unit tests, seeds the database, starts the server, runs the 70-check smoke test.
release.yml	tag v*	Runs the tests, restores the keystore from a repository secret, assembles a signed APK and App Bundle, then deletes the keystore from the runner.

Table 5: The three GitHub Actions workflows

The ordering inside each workflow is deliberate. The unit suites need no server and no emulator and finish in seconds, so they run first: if a business rule is broken there is no point spending a minute booting the API to find out.

.github/workflows/android.yml (extract)

```
jobs:
  build:
    name: Run unit tests and assemble the debug build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '17'
      - uses: gradle/actions/setup-gradle@v4
      - run: chmod +x gradlew
      - name: Run the unit tests
        run: ./gradlew testDebugUnitTest --no-daemon --stacktrace
      - name: Assemble the debug build
        run: ./gradlew assembleDebug --no-daemon --stacktrace
      - name: Upload the debug package
        uses: actions/upload-artifact@v4
        with:
```

```
name: ithrift-debug-apk
path: app/build/outputs/apk/debug/*.apk
if-no-files-found: error
```

EVIDENCE TO INSERT

A screenshot of a green Actions run showing the test and build jobs passing, and one of the run history showing repeated runs over time.

3.4 Release and deployment

Any build that clears integration is a candidate release, so releasing is a decision rather than a scramble. Tagging a commit triggers release.yml, which runs the tests again, because a tag is not a reason to skip them but the reason to be sure, and then assembles a signed APK and App Bundle.

The keystore is never committed. It is held base64-encoded in a repository secret, restored to the runner's temporary directory for the build, and deleted afterwards in a step that runs even if the build failed. This matters more than it might appear: anyone holding that file can publish an update to the Play Store under the group's name, and unlike a password it cannot be rotated once an app is live.

3.5 Monitoring and logging

The API writes one line per request, when the response finishes so that the status code and the duration are known. Failures go to stderr rather than stdout, so a CI log or a log viewer can be filtered down to just the problems. Nothing from the request body is logged, because that is where passwords and ID tokens travel.

server/index.js: request logging

```
app.use('/api', (req, res, next) => {
  const startedAt = process.hrtime.bigint();

  res.on('finish', () => {
    const ms = Number(process.hrtime.bigint() - startedAt) / 1e6;
    const who = req.user ? `${req.user.type}#${req.user.id}` : 'anonymous';
    const line = `[api] ${req.method} ${req.originalUrl} -> ${res.statusCode} ($
{ms.toFixed(1)}ms) [${who}]`;
    if (res.statusCode >= 400) console.error(line);
    else console.log(line);
  });

  next();
});
```

What that produces while the smoke test runs

```
[api] POST /api/auth/login -> 200 (86.4ms) [anonymous]
[api] GET /api/auth/profile -> 403 (1.1ms) [staff#2]
[api] PUT /api/auth/profile -> 400 (0.9ms) [customer#1]
[api] POST /api/orders -> 201 (18.7ms) [customer#13]
[api] PUT /api/admin/users/13/status -> 200 (14.0ms) [admin#1]
```

On the client, OkHttp's logging interceptor records the start and end of every network call, and the Settings, Account and Help screens log the decisions a person would want explained after the fact: a theme change, a saved profile, a rejected sign-in. Logging is included deliberately: the module asks for evidence that the team understands its own code, and a log line at each boundary is what makes a failure explainable rather than mysterious.

4. The Android application

The application is written in Kotlin and targets mid-range Android handsets, which are what most South African customers carry. It runs on a physical phone rather than only on an emulator, which is what the demonstration video shows.

Setting	Value	Why
Language	Kotlin	The supported language for Android; null-safety removes a whole class of crash.
minSdk	24 (Android 7.0)	Reaches older handsets that are still common second-hand, which is the same market the shop sells into.
targetSdk / compileSdk	35	Current platform behaviour and the Play Store requirement.
View layer	View binding	Compile-time-checked view references; no findViewById, no cast errors at runtime.
Networking	Retrofit 2 + OkHttp + Gson	A typed interface over the same REST API the website calls.
Concurrency	Kotlin coroutines	Network calls off the main thread without callback nesting.
Images	Coil (with SVG support)	Loads product images straight from the API, including generated SVG placeholders.
Sign-in	Credential Manager + Google ID	The supported single sign-on route; GoogleSignInClient is deprecated.

Table 6: The application's technical configuration and the reason for each choice

4.1 Architecture

The app uses a single-activity design. One activity hosts a fragment container and a bottom navigation bar with five tabs; every other screen opens on top of the current tab through the fragment back stack, so the system back button behaves as a person expects without any custom handling.

Screen state is held in the fragment and network access goes through a shared, typed API client rather than through a per-screen ViewModel. This is worth stating plainly, because it is a departure from the Model-View-ViewModel pattern that Google's architecture guide recommends. The reason is that the logic worth extracting from a screen in this app is not state management, since the screens are thin and each one loads a list and renders it, but validation, parsing and formatting, and those have been extracted into framework-free objects that the unit tests exercise directly. The cost of the decision is honest: rotating the device re-fetches rather than restoring from memory. Introducing ViewModels is the first refactor to make if the screens grow.

Layer	Responsibility	Example types
Activity	Hosts the fragment container, the toolbar and the bottom navigation	MainActivity
Fragment	Renders state, captures input, holds no business rules	ShopFragment, SettingsFragment, AccountFragment
Data / API client	HTTP calls, session token, typed results	ApiClient, ApiService, safeApiCall
Session and settings	The signed-in token and the device preferences	SessionManager, AppSettings
Utilities	Validation, URL parsing, formatting; framework-free and unit-tested	Validators, ApiUrl, formatMoney

Table 7: Layers of the Android application and their responsibilities

4.2 Program modules and project structure

The code is organised by feature, so a screen and the classes that support it sit together. This is what a marker will see in the repository.

app/src/main/java/com/codecuture/ithrift/

MainActivity.kt	single-activity shell: toolbar, container, bottom nav
IThriftApplication.kt	applies the saved theme before the first screen draws
data/	
ApiClient.kt	builds Retrofit/OkHttp against the configured server
ApiService.kt	the typed REST interface
ApiResponse.kt	safeApiCall(): every failure becomes a message
Models.kt	the JSON shapes the API returns
SessionManager.kt	the signed-in token and user
AppSettings.kt	theme, notifications, server address
GoogleSignIn.kt	Credential Manager -> a Google ID token
LookupCache.kt	brands and categories, fetched once
ui/	
BaseFragment.kt	shared helpers every screen uses
shop/ search/ product/	cart/ checkout/ orders/
account/	AccountFragment, SettingsFragment, HelpFragment
util/	
Validators.kt	every input rule, in one tested place
ApiUrl.kt	server address -> a URL Retrofit accepts
Extensions.kt	money, dates, star ratings, status colours

4.3 The graphical user interface

The five bottom-navigation tabs, Shop, Search, Cart, Orders and Account, are the only top-level destinations, so a customer is never more than two taps from the catalogue. The Cart tab carries a badge with the number of items in it.

The catalogue uses a two-column grid. One column wastes the screen and three make the photographs too small to judge condition, and condition is the deciding factor when buying second-hand clothing: a shopper is looking for the mark that the description mentions. Every product tile therefore leads with the photograph and carries the condition grade on it.

Dark theme

The app follows the phone's light and dark setting, and the customer can override it in Settings. This was implemented by redefining the colour palette in res/values-night/colors.xml rather than by touching any layout. Every layout already referred to the palette by name, so one file gives the whole app a dark theme and no screen can be left behind on a colour somebody hard-coded.

The dark values are not a straight inversion of the light ones. Pure white text on pure black glares and smears on an OLED panel, so the background is a very dark grey and the text is a warm off-white, and the status colours are lightened to hold their contrast against the darker ground. The theme is applied in Application.onCreate rather than in the activity, so the app opens in the right theme instead of flashing light and then going dark.

EVIDENCE TO INSERT

Clear, labelled screenshots of: Shop grid, Search with filters, Product detail, Cart, Checkout, Order confirmation, Orders list, Order detail with the tracker, Account, Settings, Help.

One pair of the same screen in light and in dark, side by side, to show the theme working.

Label each with the screen name and the tab it lives under.

4.4 Input controls and validation

Every input control is validated before it does any work, and the app is built so that a bad input produces a helpful message rather than a crash. The rules live in one object, `Validators`, rather than being written inline in each screen, so "what counts as a valid password" is answered in exactly one place, and that place matches the server's own rule. When the two disagree, the customer is told their password is fine and then rejected by the API, which is the most frustrating kind of error there is.

`Validators` deliberately touches no Android type. That is what lets the unit tests run on a plain JVM in seconds on the CI runner, with no emulator. Every function returns null when the value is acceptable, or the message to show against the field when it is not.

`app/src/main/java/com/codecuture/ithrift/util/Validators.kt` (extract)

```
object Validators {

    /** Mirrors isPasswordStrong() in server/utills/password.js. Keep the two in step. */
    fun password(value: String): String? = when {
        value.isEmpty()           -> "Enter a password."
        value.length < 8         -> "Password must be at least 8 characters
    long."
        !value.any { it.isUpperCase() } -> "Password must include an uppercase letter."
        !value.any { it.isDigit() }     -> "Password must include a number."
        else                         -> null
    }

    fun quantity(value: String, inStock: Int): String? {
        val number = value.trim().toIntOrNull()
        ?: return "Enter a quantity as a whole number."
        return when {
            number < 1           -> "Order at least one."
            inStock <= 0         -> "That item is out of stock."
            number > inStock     -> "Only $inStock in stock."
            else                 -> null
        }
    }
}
```

Network calls cannot crash a screen either. Every call goes through `safeApiCall`, which turns a network failure, a non-2xx response and a missing body all into a displayable message. Nothing above that layer ever sees an exception, which is why an unreachable server shows a retry message instead of closing the app.

`app/src/main/java/com/codecuture/ithrift/data/ApiResponse.kt` (extract)

```
sealed class ApiOutcome<out T> {
    data class Success<T>(val data: T) : ApiOutcome<T>()
    data class Failure(val message: String) : ApiOutcome<Nothing>()
}

suspend fun <T> safeApiCall(block: suspend () -> Response<T>): ApiOutcome<T> {
    return try {
        val response = block()
        if (response.isSuccessful) {
            val body = response.body()
            if (body != null) ApiOutcome.Success(body)
            else ApiOutcome.Failure("Empty response from server.")
        } else {
            ApiOutcome.Failure(parseResponseBody(response.errorBody()))
        }
    } catch (e: IOException) {
        ApiOutcome.Failure("Couldn't reach the server. Check the server address " +
            "in Account, and make sure npm start is running.")
    } catch (e: Exception) {
        ApiOutcome.Failure(e.message ?: "Something went wrong.")
    }
}
```

parseErrorBody reads the API's own { "error": "..." } body, so the message the customer sees is the one the server wrote: "Only 2 of this item is in stock." rather than "HTTP 409". Seven unit tests cover this one function, because every error message in the app passes through it.

4.5 Settings

The Account tab leads to a Settings screen, which is where the module's requirement that "the user must be able to change their settings in the app" is met. The sections are grouped by what the setting belongs to, because that is how a customer thinks about it.

Section	What the customer can change	Where it is stored
Your details	First and last name, phone number, delivery address, city, postal code	The account, through PUT /api/auth/profile
Password	The account password; the current one is required first	The account, through POST /api/auth/change-password
Appearance	Light, dark, or follow the system setting	This phone (SharedPreferences)
Notifications	Order updates; new arrivals and sales	This phone (SharedPreferences)
Connection	The server address, with a Test connection button	This phone (SharedPreferences)
Reset	Clear cached data; sign out	Nothing is stored

Table 8: The settings screen, and where each setting is kept

The split between the two storage locations is the point of the design rather than an implementation detail. A delivery address kept in SharedPreferences would mean a customer who changed it on the website still saw the old one on the phone; an appearance setting kept in the database would follow them onto a friend's phone. Account settings go to the database through the API, and device settings stay on the device.

Three smaller decisions are worth recording:

- The Password section is removed entirely for an account created through single sign-on. The API reports canChangePassword, and offering the section and then rejecting it would be a dead end.
- Test connection calls the API and says plainly what came back. Almost every "it isn't working" turns out to be the wrong server address, and this turns a mystery into a sentence.
- Clear cached data does not sign the customer out. Clearing a cache and losing a session are different things, and a button that does both surprises people.

EVIDENCE TO INSERT

A screenshot of the Settings screen showing the sections above, and one of the profile after a change has been saved.

A screenshot of Test connection reporting success, and one of it reporting a failure. The second is the more useful of the two.

5. Authentication, roles and security

5.1 Registration and sign-in

A customer can register and sign in from the Account tab. Registration validates the name, the email address and the password, creates the account through the API and signs the customer straight in. On success the API returns

a bearer token, which the client stores and attaches to every request that follows, so the account behaves identically on the app and on the website.

There are three routes into a session, register, sign in and Google sign-in, and all three end in the same place: an API-issued token in SessionManager. Nothing above that point needs to know which route was taken, which is what keeps the rest of the app free of "was this a Google user?" checks.

Sign-in failures return the same message whether the email address or the password was wrong. That is deliberate: distinguishing them would turn the sign-in form into a way of discovering which email addresses have accounts.

5.2 Password encryption

Passwords are protected in transit and at rest, as the module requires. Traffic runs over HTTPS in production, so credentials are encrypted in transit. At rest, passwords are never stored: what is stored is a salted hash, computed on the server with `scrypt`, which is a deliberately slow, memory-hard key-derivation function. Slowness is the feature: it is what makes guessing a stolen hash expensive rather than instant.

Decision	What was done	Why
Algorithm	<code>scrypt</code> (Node's built-in <code>node:crypto</code>)	Memory-hard and deliberately slow, so a stolen database is expensive to attack. No third-party dependency.
Salt	16 random bytes per account	Two customers who choose the same password get different hashes, so cracking one does not crack both.
Key length	64 bytes	Long enough that collisions are not a practical concern.
Comparison	<code>crypto.timingSafeEqual</code>	A normal comparison returns faster on a near-miss, which leaks the password one character at a time.
Where it runs	On the server, never the client	A client can be repackaged; whatever it computes can be forged.

Table 9: How passwords are stored, and the reasoning behind each choice

`server/utils/password.js`

```
const crypto = require('node:crypto');
const KEY_LEN = 64;

function hashPassword(password) {
  const salt = crypto.randomBytes(16).toString('hex');
  const hash = crypto.scryptSync(password, salt, KEY_LEN).toString('hex');
  return { hash, salt };
}

function verifyPassword(password, salt, expectedHashHex) {
  const candidate = crypto.scryptSync(password, salt, KEY_LEN);
  const expected = Buffer.from(expectedHashHex, 'hex');
  if (candidate.length !== expected.length) return false;
  // Constant-time: a normal === returns early on the first differing byte.
  return crypto.timingSafeEqual(candidate, expected);
}
```

Five unit tests cover this file directly. They assert that a hash never contains the plain text, that the same password hashes differently for two accounts, that a wrong password and a wrong salt both fail, and that the policy accepts strong passwords and rejects short, lowercase-only and digit-less ones.

5.3 Single sign-on

Single sign-on with Google is offered so that a customer can use an identity they already have rather than inventing another password on a phone keyboard. It is implemented end to end and, more importantly, implemented in the only way that is actually safe.

The app's entire job in single sign-on is to ask Android for a Google ID token and hand it to the API. It deliberately does not decide who the customer is, because it cannot: anything the app worked out locally could be forged by anyone who repackaged it. The API verifies the token with Google and only then issues a session of its own.

Step	Where	What happens
1	Android app	Credential Manager shows the Google account chooser and returns a signed ID token.
2	Android app	The token is POSTed to /api/auth/sso. The app never inspects it.
3	API	The token is verified with Google: signature, issuer, audience, expiry, and that the email address is verified.
4	API	The customer is matched on the provider's subject, or created if they are new.
5	API	An iTHRIFT session token is issued, the same kind a password sign-in produces.

Table 10: The single sign-on flow, and which side does each part

Four checks in that verification are each there for a specific reason:

- The audience is checked against our own client id. A validly signed Google token issued to somebody else's application must not be accepted here.
- The email address must be marked verified. Otherwise anyone could sign up to Google with someone else's address, never prove they own it, and claim the matching iTHRIFT account.
- Matching is on the provider's subject, which is Google's immutable user id, rather than on the email address, because a person can change the address on their Google account.
- If the address already belongs to a password account, the sign-in is refused rather than linked. Linking automatically would let anyone who could create a Google account on that address walk into the existing one.

server/routes/auth.js: the SSO route (extract)

```
router.post('/sso', async (req, res) => {
  const { idToken } = req.body || {};

  let identity;
  try {
    identity = await verifyGoogleIdToken(idToken);
  } catch (err) {
    console.warn('[auth] single sign-on rejected:', err.message);
    return res.status(401).json({ error: err.message });
  }

  const db = getDb();
  // Match on the provider subject: it never changes, an email address can.
  let customer = db.prepare('SELECT * FROM Customer WHERE ProviderSubject = ?')
    .get(identity.subject);

  if (!customer) {
    const byEmail = db.prepare('SELECT * FROM Customer WHERE Email = ?')
      .get(identity.email);
    if (byEmail && byEmail.AuthProvider === 'password') {
      return res.status(409).json({ error:
        'An account with that email already exists. Sign in with your email and password
        instead.' });
    }
  }
});
```



```

    }
    customer = byEmail;
  }
  // ... create the customer and their cart if still not found, then:
  const user = { type: 'customer', id: customer.CustomerID,
    name: `${customer.FirstName} ${customer.LastName}` };
  res.json({ token: createSession(user), user });
});

```

Configuration, and why the button can be hidden

Single sign-on needs an OAuth web client id from a Google Cloud project. Neither the client id nor anything derived from it is committed: the app reads it from local.properties, which is git-ignored, and the server reads it from the GOOGLE_CLIENT_ID environment variable. A checkout on a clean machine, or on the CI runner, still builds. It simply builds without single sign-on.

When it is not configured, the app hides the button rather than offering one that fails. It checks two things: whether this build has a client id at all, and whether the server it is pointed at reports single sign-on as enabled (GET /api/auth/sso/status). A button that always fails is worse than no button. Equally, an unconfigured server refuses every token rather than accepting any, because a server that trusted tokens minted for any application at all would be worse than one with no SSO.

EVIDENCE TO INSERT

A screenshot of the sign-in screen with the Continue with Google button, and one of a successful Google sign-in.

A screenshot of the account in the Google Cloud console or the identity provider, showing the stored identity. The module asks the video to show what data is held in the online authentication service.

Name the identity provider you used in the video voice-over.

5.4 Roles and server-side access control

There are three roles: customer, staff and administrator. The Android application contains no administrative screens at all, and a staff account that signs in is told to use the website. Every request is checked on the server.

This is the difference between a convenience and a control. Hiding a menu item stops an honest person from clicking something they should not; the role check on the API is what stops a dishonest one from typing the address by hand. Both exist here, and only the second is relied on.

server/middleware/auth.js: the role check

```

function requireRole(...types) {
  return (req, res, next) => {
    if (!req.user) return res.status(401)
      .json({ error: 'Sign in is required for this action.' });
    if (!types.includes(req.user.type)) {
      return res.status(403).json({ error: 'You do not have permission to do that.' });
    }
    next();
  };
}

// applied per route, e.g.:
router.get('/profile', requireAuth, requireRole('customer'), ...);
router.put('/orders/:id/status', requireRole('staff', 'admin'), ...);

```

The profile and settings routes take the customer id from the session and never from the request body. That is what stops a customer editing somebody else's details by changing a number on the phone. Eleven of the seventy end-to-end checks exist purely to confirm that each role is refused what it should be refused.

6. Database design

6.1 Database management system

The design specifies MySQL 8, and the production schema is in the repository at database/mysql-schema.sql. The prototype runs on SQLite through Node's built-in node:sqlite module, so that the whole system starts with one command and no database server to install, which matters when the marker has to run it.

The table structure, the keys and the Third Normal Form design are identical across the two. One difference is recorded honestly: SQLite treats ORDER as a reserved word, so that table is named Orders, and the MySQL schema keeps the same name for consistency between them.

6.2 Tables and relationships

Eleven tables, in Third Normal Form, carried forward from the XISD5319 system design and extended in Task 2 to record how an account authenticates.

Table	Purpose	Key relationships
Brand	Brand names, held once	PK BrandID
Category	Category names, held once	PK CategoryID
Product	Catalogue items: size, condition, price, stock	FK BrandID, CategoryID
Customer	Customer accounts, credentials and addresses	PK CustomerID
Admin	Staff and administrator accounts	PK AdminID
Cart	One open cart per customer	FK CustomerID (unique)
CartItem	A line in a cart	FK CartID, ProductID
Orders	A placed order, its status and total	FK CustomerID
OrderItem	A line in an order, with the price paid	FK OrderID, ProductID
Payment	One payment per order	FK OrderID (unique)
Review	A star rating and comment on a product	FK ProductID, CustomerID

Table 11: The eleven tables and their relationships (Third Normal Form)

Two design points are worth drawing out. Brand and Category are separate tables rather than text columns on Product, which is what makes the filter chips a lookup rather than a scan of every row, and what stops "Adidas" and "adidas" existing side by side. OrderItem stores UnitPrice rather than reading it from Product, because the price a customer paid is a historical fact: repricing an item must not silently rewrite what an old order was worth.

6.3 Recording how an account signs in

Task 2 added single sign-on, and the Customer table had to change to record it. The password columns became nullable, because a customer who signs in through Google never chooses a password on this system, and two columns were added: AuthProvider, and ProviderSubject holding the provider's own immutable user id.

Making the password columns nullable creates an obvious hole, because a row could now exist with no credentials of any kind, so the table carries a constraint that closes it. A password account must have a hash and a salt; an SSO account must have a provider subject. The database refuses anything else, which means the rule holds even if a future route forgets to check.

server/init-db.js: the Customer table

```

CREATE TABLE Customer (
  CustomerID      INTEGER PRIMARY KEY AUTOINCREMENT,
  FirstName       TEXT NOT NULL,
  LastName        TEXT NOT NULL,
  Email           TEXT NOT NULL UNIQUE,
  PasswordHash    TEXT,
  PasswordSalt    TEXT,
  AuthProvider    TEXT NOT NULL DEFAULT 'password'
                  CHECK (AuthProvider IN ('password','google')),
  ProviderSubject TEXT UNIQUE,
  Phone           TEXT,
  AddressLine     TEXT,
  City            TEXT,
  PostalCode      TEXT,
  Status          TEXT NOT NULL DEFAULT 'active'
                  CHECK (Status IN ('active','suspended')),
  CreatedAt       TEXT NOT NULL DEFAULT (datetime('now')),
  -- A password account must carry a hash; an SSO account must carry a subject.
  CHECK (
    (AuthProvider = 'password' AND PasswordHash IS NOT NULL
      AND PasswordSalt IS NOT NULL)
    OR
    (AuthProvider = 'google' AND ProviderSubject IS NOT NULL)
  )
);

```

6.4 Seed data

The module requires at least ten records in every table. Every table meets that, and the seed script counts the rows and exits with an error rather than finishing quietly if any table falls short. A seed script that silently underfills a table is worse than one that fails, because the gap is only discovered during the demonstration.

Table	Rows	Table	Rows
Brand	15	CartItem	12
Category	12	Orders	12
Product	63	OrderItem	20
Customer	12	Payment	12
Admin	10	Review	15
Cart	12		

Table 12: Rows seeded per table; every one meets the ten-record minimum

The data is shaped to make the system demonstrable, not merely to reach ten. The twelve orders are spread across all four statuses and all three payment methods, and are dated from twelve weeks ago to yesterday, because the order queue, the status filter and the "last 30 days" figures on the dashboard are only worth looking at if there is something in every state. Seven customers have items left in an open cart, as real shops always do. The fifteen reviews are not all five stars: a catalogue where every rating is perfect tells a shopper nothing, and the low ratings are the honest ones for second-hand stock, because the complaint is almost always that a piece was more worn than the photographs suggested, which is exactly what the condition grade exists to prevent.

The names, addresses and phone numbers are invented. No real customer's details appear anywhere in the repository.

The seed script checks itself

```

const MINIMUM_ROWS = 10;
const tables = ['Brand', 'Category', 'Product', 'Customer', 'Admin', 'Cart',

```

```

    'CartItem', 'Orders', 'OrderItem', 'Payment', 'Review'];

for (const table of tables) {
  const { total } = db.prepare(`SELECT COUNT(*) AS total FROM ${table}`).get();
  counts[table] = total;
  if (total < MINIMUM_ROWS) shortfall.push(`${table} (${total})`);
}

if (shortfall.length > 0) {
  console.error(`\nThese tables hold fewer than ${MINIMUM_ROWS} rows: $
{shortfall.join(', ')}.\n`);
  process.exit(1);
}

```

npm run init-db

```

Creating schema...
Seeding brands and categories...
Seeding products...
Seeding customers...
Seeding staff and administrator accounts...
Seeding orders, payments and reviews...
Seeding open carts...
Seeding reviews...
Database created at ...data\ithrift.db
Seed summary: 15 Brand, 12 Category, 63 Product, 12 Customer, 10 Admin,
               12 Cart, 12 CartItem, 12 Orders, 20 OrderItem, 12 Payment,
               15 Review.

```

```

Row count per table (minimum required: 10)
sqlite> SELECT Brand AS TableName, COUNT(*) AS Rows FROM Brand UNION ALL SELECT 'Category', COUNT(*) FROM Category UNION ALL SELECT 'Product',
COUNT(*) FROM Product UNION ALL SELECT 'Customer', COUNT(*) FROM Customer UNION ALL SELECT 'Admin', COUNT(*) FROM Admin UNION ALL SELECT 'Cart',
COUNT(*) FROM Cart UNION ALL SELECT 'CartItem', COUNT(*) FROM CartItem UNION ALL SELECT 'Orders', COUNT(*) FROM Orders UNION ALL SELECT 'OrderItem',
COUNT(*) FROM OrderItem UNION ALL SELECT 'Payment', COUNT(*) FROM Payment UNION ALL SELECT 'Review', COUNT(*) FROM Review;

```

TableName	Rows
Brand	15
Category	12
Product	63
Customer	12
Admin	10
Cart	12
CartItem	12
Orders	12
OrderItem	20
Payment	12
Review	15

11 rows - data/ithrift.db

Figure 1: Rows in every table after npm run init-db; each meets the ten-record minimum

Customer table: passwords are stored only as salted script hashes

```
sqlite> SELECT CustomerID, FirstName, LastName, Email, AuthProvider, PasswordSalt, PasswordHash, City, Status FROM Customer;
```

CustomerID	FirstName	LastName	Email	AuthProvider	PasswordSalt	PasswordHash	City	Status
1	Lerato	Mokoena	lerato.m@gmail.com	password	80c530b5f5d135d446...	725d1bea9caf8ec4a46cd11def736f7f02cld193...	Pretoria	active
2	Sipho	Ndlovu	sipho.n@gmail.com	password	8d461f269e4e147219...	39e6cbb4e94debd780bd4583c4c65cd7a03c26a1...	Centurion	active
3	Amahle	Dube	amahle.d@gmail.com	password	969be6963955240518...	f9eaa7b44ec73cb4e5f4bed29823662bd3540244...	Pretoria	active
4	Thabo	Molefe	thabo.molefe@gmail.com	password	6d7282899f0cb27526...	3983c2def70731ff6c0411580e9b03a881320179...	Pretoria	active
5	Nandi	Zulu	nandi.zulu@outlook.com	password	008039d629d56c5665...	5dd6421840509c197375d3f92d40dfa64f480625...	Pretoria	active
6	Kagiso	Sithole	kagiso.s@gmail.com	password	b90f32e7f12c0b2c91...	0491525194472562cd16d8eb19a663b5edfdd13b...	Pretoria	active
7	Zanele	Mahlangu	zanele.m@gmail.com	password	436d450166a309a8b9...	9aba036b1200a5bbe386d22110207b6c911f80ad...	Centurion	active
8	Tshepo	Radebe	tshepo.r@outlook.com	password	a29fb249ed3be96352...	249f137dee0f1b0e7175641d8ba2f9f169bdfdf3...	Pretoria	active
9	Palesa	Nkosi	palesa.nkosi@gmail.com	password	4ee2a3ad56f33ffbae...	c5cb947cb43ba932281a20e1f7e3db9b3a6726f...	Pretoria	active
10	Bongani	Khoza	bongani.k@gmail.com	password	b245432c5194bfcdac...	296b77979cf7ba6adaabd9e4f94b86b5a1d36afd...	Pretoria	active
11	Refilwe	Motaung	refilwe.m@outlook.com	password	ced014d9b3c94848e9...	0a7a6f8e537cf64b0f0642ca4b0fcbd3fbc54b9a...	Pretoria	active
12	Ayanda	Mthembu	ayanda.mthembu@gmail.com	password	c331e7d4524211893d...	0025ddad1c1750b9a8e1e03fd63d655a98cb2bc...	Centurion	active

12 rows - data/ithrift.db

Figure 2: The Customer table: passwords are held only as a per-account salt and an script hash

Product table (first 15 of 63 rows)

```
sqlite> SELECT ProductID, Name, BrandID, CategoryID, Size, ConditionGrade, Price, StockQty, ImageFile FROM Product LIMIT 15;
```

ProductID	Name	BrandID	CategoryID	Size	ConditionGrade	Price	StockQty	ImageFile
1	EQT Running Shoes	1	3	UK 8	Excellent	899	2	/images/products/adidas-eqt.jpg
2	Samba OG Trainers	1	3	UK 9	Very Good	1099	2	/images/products/adidas-samba.jpg
3	Samba Suede Trainers	1	3	UK 7	Excellent	1149	2	/images/products/adidas-samba2.jpg
4	Spezial Trainers	1	3	UK 8	Good	949	1	/images/products/adidas-spezial.jpg
5	Ultraboost Sneakers	1	3	UK 9	Excellent	1399	2	/images/products/adidas-ultraboost.jpg
6	Air Force 1 Low	10	3	UK 8	Very Good	1199	3	/images/products/nike-airforcel.jpg
7	Air Max 90	10	3	UK 9	Good	999	1	/images/products/nike-airmax90.jpg
8	Air Max 90 White	10	3	UK 7	Excellent	1049	2	/images/products/nike-airmax90-2.jpg
9	Dunk Low	10	3	UK 8	Very Good	1099	2	/images/products/nike-dunklow.jpg
10	Air Jordan 1 Low	10	3	UK 9	Good	1299	1	/images/products/nike-jordan1.jpg
11	Air Jordan 1 High '85	10	3	UK 8	Excellent	1599	0	/images/products/nike-jordan1-high.jpg
12	Suede Basket Sneakers	11	3	UK 7	Very Good	749	3	/images/products/puma-basket.jpg
13	Speedcat Sneakers	11	3	UK 8	Excellent	799	1	/images/products/puma-speedcat.jpg
14	Speedcat Navy Sneakers	11	3	UK 9	Very Good	779	2	/images/products/puma-speedcat2.jpg
15	Suede Classic Sneakers	11	3	UK 8	Good	649	5	/images/products/puma-suede.jpg

15 rows · data/ithrift.db

Figure 3: The Product table, first fifteen of 63 rows, with brand and category as foreign keys

Orders joined to Payment

```
sqlite> SELECT o.OrderID, o.CustomerID, o.Status, o.TotalAmount, o.CourierRef, p.Method, p.Status AS PaymentStatus, o.CreatedAt FROM Orders o JOIN Payment p ON p.OrderID = o.OrderID;
```

OrderID	CustomerID	Status	TotalAmount	CourierRef	Method	PaymentStatus	CreatedAt
1	1	Delivered	1598	CR-1001	card	paid	2026-07-03 01:58:25
2	2	Delivered	629	CR-1002	payfast	paid	2026-07-26 01:58:25
3	4	Delivered	1438	CR-1004	card	paid	2026-08-11 01:58:25
4	5	Delivered	449	CR-1005	eft	paid	2026-08-18 01:58:25
5	6	Delivered	1547	CR-1006	payfast	paid	2026-08-29 01:58:25
6	7	Cancelled	299	NULL	eft	pending	2026-09-03 01:58:25
7	8	Delivered	978	CR-1008	card	paid	2026-09-07 01:58:25
8	9	Shipped	878	CR-1009	payfast	paid	2026-09-14 01:58:25
9	10	Shipped	1099	CR-1010	card	paid	2026-09-18 01:58:25
10	3	Processing	2148	NULL	eft	pending	2026-09-22 01:58:25
11	11	Processing	678	NULL	card	paid	2026-09-23 01:58:25
12	12	Processing	799	NULL	payfast	paid	2026-09-24 01:58:25

12 rows · data/ithrift.db

Figure 4: Orders joined to Payment: twelve orders across all four statuses and three payment methods

7. The shared REST API

The API is the single application tier for both clients. It exposes JSON over HTTP, enforces every business rule, and is the only tier that can reach the database. The app and the website consume exactly the same routes, which is what makes "one database, two clients" true rather than merely claimed.

Method and path	Purpose	Access
POST /api/auth/register	Create a customer account	Public
POST /api/auth/login	Sign in; returns a bearer token	Public
POST /api/auth/sso	Sign in with a Google ID token	Public
GET /api/auth/sso/status	Whether SSO is configured on this server	Public
GET /api/auth/profile	The signed-in customer's own details	Customer
PUT /api/auth/profile	Update those details	Customer
POST /api/auth/change-password	Change the password	Customer
GET /api/products	Catalogue, with search and filters	Public

Method and path	Purpose	Access
GET /api/products/:id	Product detail	Public
GET /api/cart	The current cart	Customer
POST /api/cart/items	Add to the cart	Customer
POST /api/orders	Check out: lock stock and place the order	Customer
GET /api/orders	The customer's orders (all orders, for staff)	Customer / Staff
PUT /api/orders/:id/status	Advance an order's status	Staff / Admin
GET /api/admin/users	Customer accounts	Admin
GET /api/admin/reports/sales	Sales reporting	Staff / Admin

Table 13: The main REST API routes shared by both clients (routes added in Task 2 shown in the auth block)

The most important rule in the whole system is checkout. The order total is recalculated on the server from the current product prices, and the stock is locked in the same database transaction. That is what stops two customers buying the same one-off item, and it is why the client's idea of the total is ignored entirely: a client that could set its own total could set it to one rand.

8. Website updates

The website was updated to consume the new REST API and to match the Task 1 wireframes. The storefront (catalogue, search, product detail, cart, checkout and order tracking) and the staff and administrator console (dashboard, listings, inventory, order queue, customers and reports) now run through the same shared API as the application rather than through server-rendered pages.

The administrator-only Customers section is rendered for administrators alone, and the server refuses the request from any other role even if the address is typed by hand. There is also an installable Progressive Web App build of the storefront at /mobile, which is the fallback for anyone who cannot install the Android application.

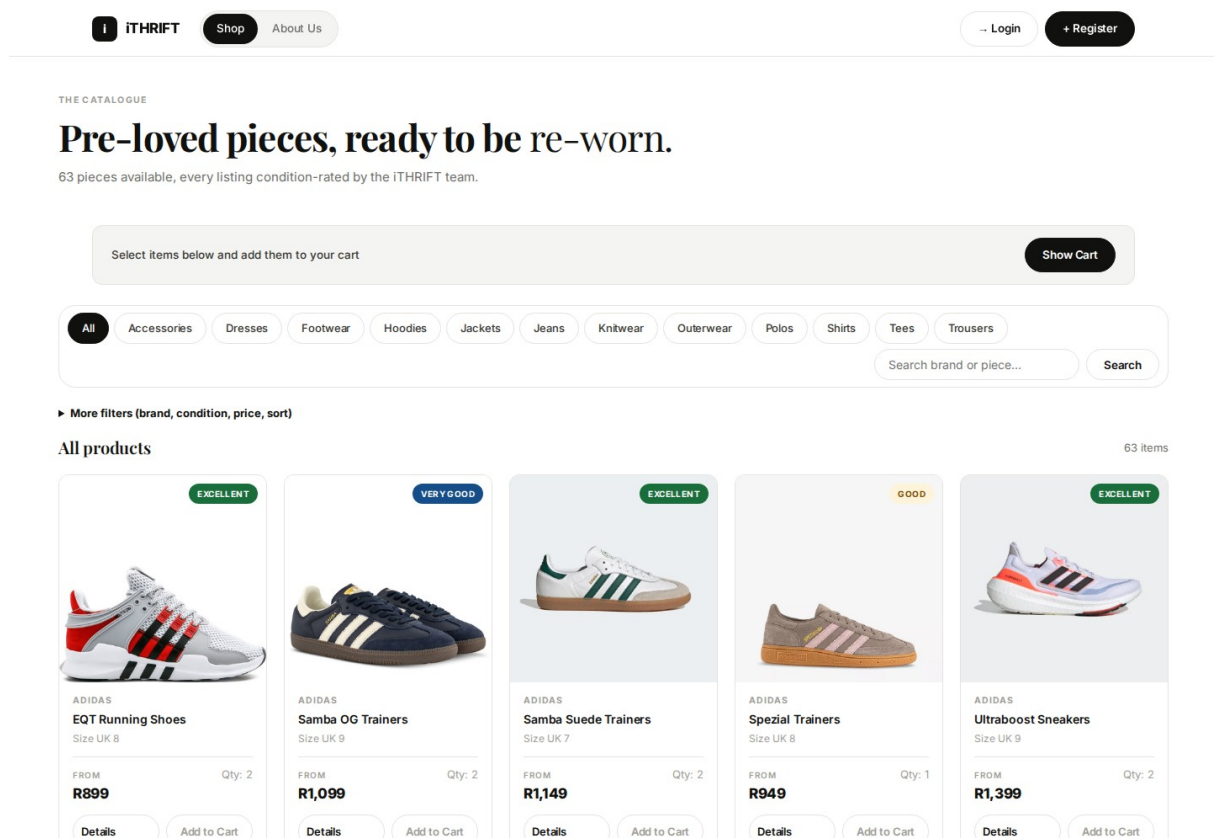


Figure 5: Website, Shop: the catalogue with category filters and condition grades

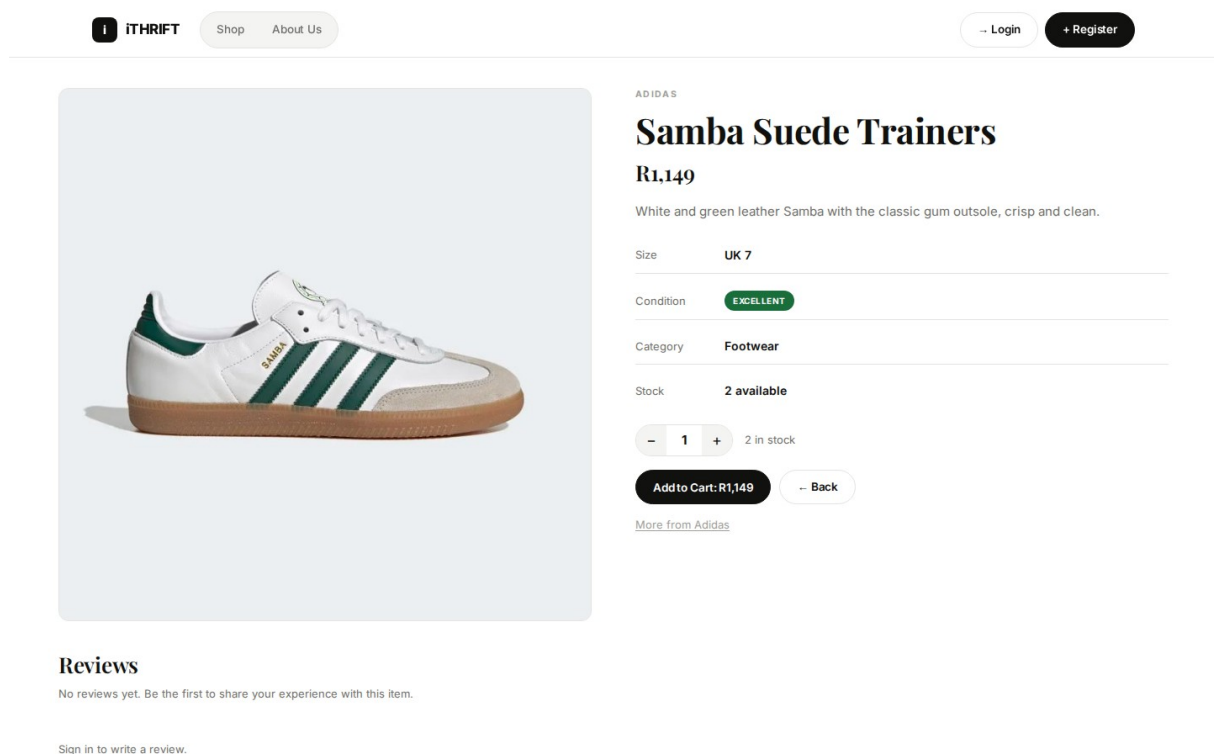


Figure 6: Website, Product detail: size, condition, stock and reviews

[Shop](#)
[About Us](#)

[My Orders](#)
[Cart 2](#)
[Account](#)

[Back to orders](#)

ORD-0001

Placed 09 Jul 2026 by Lerato Mokoena (lerato.m@gmail.com)

Processing

Shipped

Delivered

Items

1 × Air Force 1 Low (PRD006)	R1,199
1 × Slim Chino Trousers (PRD048)	R399
Total	R1,598
Payment	CARD · paid

Delivery address

Lerato Mokoena

This prototype simulates courier delivery; no live courier integration is included.

Pre-loved branded fashion: condition-rated, seller-verified and affordably priced for South African shoppers.

HELP

- [Help Centre](#)
- [Selling & Buying](#)
- [Staying Safe](#)

LEGAL BITS

- [Terms & Conditions](#)
- [Privacy Policy](#)
- [Shipping Policy](#)
- [Dispute Policy](#)

ACCOUNT

- [Sign in](#)
- [Register](#)
- [My Orders](#)
- [Admin portal](#)

Figure 7: Website, My Orders: order tracking with the courier reference

[Shop](#)
[About Us](#)
[Admin](#)

Signed in as Naledi Khumalo **ADMIN**
[Account](#)

[Dashboard](#)
[Listings](#)
[Inventory](#)
[Process orders](#)
[Customers](#)

Dashboard

R12,241
TOTAL REVENUE

11
ORDERS PLACED

1
CANCELLED

6
DELIVERED

3
PROCESSING

2
SHIPPED

Revenue by brand

BRAND	UNITS SOLD	REVENUE
Nike	4	R4,896
Generic	8	R2,832
Calvin Klein	2	R1,148
Adidas	1	R1,099
Tommy Hilfiger	2	R968
Puma	1	R799
Zara	1	R499

Revenue by category

CATEGORY	UNITS SOLD	REVENUE
Footwear	6	R6,794
Knitwear	3	R1,557
Trousers	3	R1,187
Dresses	2	R948
Hoodies	1	R599
Tees	2	R538
Jeans	1	R349
Polos	1	R269

Figure 8: Website, Admin console: dashboard with revenue by brand and category

ITHRIFT

ShopAbout UsAdmin

Signed in as Naledi KhumaloADMINAccount

Dashboard

Listings

Inventory

Process orders

Customers

Process orders

All statuses

ORDER	CUSTOMER	DATE	STATUS	TOTAL	
ORD-0012	Ayanda Mthembu	yesterday	PROCESSING	R799	Open
ORD-0011	Refilwe Motaung	2 days ago	PROCESSING	R678	Open
ORD-0010	Amahle Dube	3 days ago	PROCESSING	R2,148	Open
ORD-0009	Bongani Khoza	7 days ago	SHIPPED	R1,099	Open
ORD-0008	Palesa Nkosi	11 days ago	SHIPPED	R878	Open
ORD-0007	Tshepo Radebe	18 days ago	DELIVERED	R978	Open
ORD-0006	Zanele Mahlangu	22 days ago	CANCELLED	R299	Open
ORD-0005	Kagiso Sithole	27 days ago	DELIVERED	R1,547	Open
ORD-0004	Nandi Zulu	24 Aug 2026	DELIVERED	R449	Open
ORD-0003	Thabo Molefe	17 Aug 2026	DELIVERED	R1,438	Open
ORD-0002	Sipho Ndlovu	01 Aug 2026	DELIVERED	R629	Open
ORD-0001	Lerato Mokoena	09 Jul 2026	DELIVERED	R1,598	Open

Figure 9: Website, Admin console: the order queue that staff work through

9. Automated testing

There are 128 automated checks in three suites, and all three run through GitHub Actions so that a change is tested on the pipeline and not only on one developer's machine. The module asks for exactly this: proof that the code works somewhere other than the computer it was written on.

Suite	Checks	Runs with	What it covers
Kotlin unit tests app/src/test/	49	./gradlew testDebugUnitTest	Input validation, server-address parsing, money and date formatting, network error handling, the theme setting
API unit tests test/unit-tests.js	9	npm run test:unit	Password hashing and verification, the password policy, SSO configuration, cart arithmetic
End-to-end smoke test test/smoke-test.js	70	npm test	The whole API surface, against a running server and a seeded database

Table 14: The three test suites

The division is deliberate. A unit test needs no server, no database and no network, so it runs in seconds and points straight at the function that broke. An end-to-end test proves the parts work together but, when it fails, only tells you that a request returned the wrong status. Having both means a failure is usually diagnosed by reading the test name.

9.1 Kotlin unit tests

This suite is new in Task 2, and it closed a real hole. The Android workflow already ran `./gradlew testDebugUnitTest`, but `app/src/test/` did not exist, so the step passed on every run without testing anything at all. A green tick that means nothing is worse than a red one.

Test class	Tests	What it pins down
ValidatorsTest	19	Every input rule: email, password policy, required fields, quantity against stock, postal code, phone number
SafeApiCallTest	7	That no network failure can reach a screen as an exception
FormattingTest	10	Rand formatting, star ratings, relative dates
ApiUrlTest	8	Server-address parsing: missing scheme, trailing slashes, whitespace, blank
ThemeModeTest	5	That the appearance setting survives storage and upgrades

Table 15: The 49 Kotlin unit tests

Two of these are worth explaining, because they guard against bugs that would only appear on a customer's phone.

FormattingTest: money must not follow the phone's locale

```
@Test
fun `thousands are grouped with a comma regardless of the phone's locale`() {
    val original = Locale.getDefault()
    try {
        // A locale whose own convention is a space or a full stop.
        Locale.setDefault(Locale.FRANCE)
        assertEquals("R1,620", formatMoney(1620.0))
        assertEquals("R12,500", formatMoney(12500.0))
    } finally {
        Locale.setDefault(original)
    }
}
```

On a phone set to Afrikaans or French the default grouping separator is a space or a full stop, and "R1.620" reads as one rand sixty-two to half the people who see it. Pinning the output means a well-meaning change to the locale fails a test instead of quietly mispricing the catalogue.

ThemeModeTest: the setting must survive an upgrade

```
@Test
fun `the stored values are the exact strings written to preferences`() {
    // Pinned literally: changing one of these would orphan the preference
    // already saved on every installed copy of the app.
    assertEquals("system", AppSettings.ThemeMode.SYSTEM.stored)
    assertEquals("light", AppSettings.ThemeMode.LIGHT.stored)
    assertEquals("dark", AppSettings.ThemeMode.DARK.stored)
}
```

The theme is stored as a string rather than as an enum ordinal, and this test is what keeps it that way. Storing an ordinal would mean that inserting a fourth theme into the middle of the enum silently re-interprets everyone's saved preference as a different one, which is a bug that only surfaces on an upgrade, on a customer's phone, where nobody is watching.

9.2 API unit tests

Nine checks on the API's pure logic, using `node:test`, which is built into Node and needs no framework. Five of them cover password storage, which is the rule with the highest cost of failure in the system.

test/unit-tests.js (extract)

```
test('the same password hashes differently for two accounts', () => {
  // Without a per-account salt, two customers who picked the same password
  // would share a hash, and cracking one would crack both.
  const a = hashPassword('Password1');
  const b = hashPassword('Password1');
  assert.notEqual(a.salt, b.salt);
  assert.notEqual(a.hash, b.hash);
});

test('the correct password verifies and a wrong one does not', () => {
  const { hash, salt } = hashPassword('Password1');
  assert.equal(verifyPassword('Password1', salt, hash), true);
  assert.equal(verifyPassword('password1', salt, hash), false,
    'verification must be case sensitive');
});
```

9.3 End-to-end smoke test

Seventy checks drive the running server exactly as a client would: register, sign in, browse, filter, search, add to a cart, check out, track an order, review a product, and try each of those as the wrong role. It grew from 51 checks in Task 1 to 70 in Task 2, with the new checks covering single sign-on, the profile routes and password changes.

The single sign-on checks are worth describing, because what they test is not the happy path. A successful Google sign-in needs a real ID token, which only a person tapping the button on a handset can produce, and that is what the demonstration video is for. What is testable here, and what actually protects the system, is the refusal.

test/smoke-test.js: single sign-on

```
const forged = await call('/api/auth/sso', { method: 'POST',
  body: { idToken: 'not.a.real.token' } });
check('SSO refuses a token it has not verified with Google', forged.status === 401);

const empty = await call('/api/auth/sso', { method: 'POST', body: {} });
check('SSO refuses a request with no token at all', empty.status === 401);
```

One more check is worth pointing out, because it is the kind of thing that is easy to get wrong and impossible to see by looking at a screen:

The profile route must never leak the hash

```
check('Profile never returns the password hash',
  !JSON.stringify(profile.data).toLowerCase().includes('passwordhash'));
```

9.4 Results

All three suites pass. The two API suites were re-run on 25 September 2026 against a freshly seeded database, after the Task 1 feedback changes described in Section 1.5, and both passed in full. The Kotlin figures are from the last local Gradle run on 7 September 2026; GitHub Actions re-runs that suite on every push, which is the evidence that it passes on a clean machine rather than only on ours.

Suite	Result
Kotlin unit tests	49 passed, 0 failed
API unit tests	9 passed, 0 failed
End-to-end smoke test	70 passed, 0 failed, 70 total checks

Suite	Result
Debug build	BUILD SUCCESSFUL. app-debug.apk, about 8 MB

Table 16: Test and build results

EVIDENCE TO INSERT

A screenshot of a GitHub Actions run showing the android.yml and api-and-website.yml jobs passing, and one of the run history. These appear once the Task 2 commits are pushed.

```

API unit tests (node:test)

$ npm run test:unit
# Subtest: a hashed password never contains the plain text
ok 1 - a hashed password never contains the plain text
# Subtest: the same password hashes differently for two accounts
ok 2 - the same password hashes differently for two accounts
# Subtest: the correct password verifies and a wrong one does not
ok 3 - the correct password verifies and a wrong one does not
# Subtest: a password verified against the wrong salt fails
ok 4 - a password verified against the wrong salt fails
# Subtest: the password policy accepts strong passwords and rejects weak ones
ok 5 - the password policy accepts strong passwords and rejects weak ones
# Subtest: product and order references are zero padded to a fixed width
ok 6 - product and order references are zero padded to a fixed width
# Subtest: single sign-on is off unless a client id is configured
ok 7 - single sign-on is off unless a client id is configured
# Subtest: the cart subtotal multiplies each line by its quantity
ok 8 - the cart subtotal multiplies each line by its quantity
# Subtest: an empty cart totals zero rather than failing
ok 9 - an empty cart totals zero rather than failing
# tests 9
# suites 0
# pass 9
# fail 0
# cancelled 0
# skipped 0
# todo 0
# duration_ms 601.868844

```

Figure 10: API unit tests: 9 passed, 0 failed (25 September 2026)

```

End-to-end smoke test against the running API and seeded database

$ npm test

Running iTHRIFT Clothes smoke test against http://localhost:3000

✓ API root responds with status ok
✓ Registration rejects a weak password
✓ Registration succeeds with a valid password
✓ Registration rejects a duplicate email
✓ Login rejects an incorrect password
✓ Seeded customer (Lerato) can log in
✓ Administrator can log in
✓ Staff member can log in
✓ GET /auth/me reflects the signed-in customer
✓ SSO status endpoint reports whether the server is configured
✓ SSO refuses a token it has not verified with Google
✓ SSO refuses a request with no token at all
✓ Profile requires authentication (401 without a token)
✓ Profile is a customer route; staff are refused (403)
✓ Customer can read their own profile
✓ Profile says whether the account has a password to change
✓ Profile never returns the password hash
✓ Profile update rejects a postal code that is not four digits
✓ Profile update rejects a blank first name
✓ Customer can update their own profile
✓ The updated phone number was actually persisted
...
✓ A customer can submit a product review
✓ The submitted review appears in the product's review list
✓ A customer cannot view the sales report (403)
✓ Staff can view the sales report
✓ Inventory report flags the seeded out-of-stock product
✓ User management is reserved for the administrator (staff gets 403)
✓ Administrator can list customer accounts
✓ Administrator can suspend a customer account
✓ A suspended customer cannot log in
✓ Administrator can reactivate a suspended account
✓ Logout succeeds
✓ The token no longer works after logout

70 passed, 0 failed, 70 total checks.

```

Figure 11: End-to-end smoke test: 70 passed, 0 failed (25 September 2026)

```

API request log written while the smoke test ran

$ npm start
[api] POST /api/products/1/reviews -> 201 (1.6ms) [customer#13]
[api] GET /api/products/1/reviews -> 200 (0.4ms) [anonymous]
[api] GET /api/admin/reports/sales -> 403 (0.2ms) [customer#13]
[api] GET /api/admin/reports/sales -> 200 (0.6ms) [staff#2]
[api] GET /api/admin/reports/inventory -> 200 (0.6ms) [admin#1]
[api] GET /api/admin/users -> 403 (0.3ms) [staff#2]
[api] GET /api/admin/users -> 200 (0.4ms) [admin#1]
[api] PUT /api/admin/users/13/status -> 200 (1.8ms) [admin#1]
[api] POST /api/auth/login -> 403 (41.9ms) [anonymous]
[api] PUT /api/admin/users/13/status -> 200 (1.8ms) [admin#1]
[api] POST /api/auth/logout -> 200 (0.3ms) [customer#13]
[api] GET /api/cart -> 401 (0.2ms) [anonymous]

```

Figure 12: The API request log written during that run, showing refused requests logged as failures

10. Verification and validation

Beyond the automated suites, the team ran a manual pass focused on invalid input and error handling, because the module requires that a bad input never crashes the application. The table below is the test log. Each case names the automated check that now covers it, where one exists, because a manual test that finds a bug should become an automated test or it will be found again.

#	Test case	Expected result	Covered by
1	Register with a malformed email address	Inline message, no account created	ValidatorsTest, smoke test
2	Register with a weak password	Inline message naming the rule that failed	ValidatorsTest, unit tests
3	Sign in with the wrong password	Generic "incorrect email or password"	Smoke test
4	Sign in to a Google account with a password	Told to use the Google button instead	Manual
5	Add more items than are in stock	Blocked with "Only n in stock."	ValidatorsTest, smoke test
6	Two customers buy the last item	Only the first succeeds	Smoke test
7	Check out with no network	Message with advice, no crash	SafeApiCallTest
8	Point the app at a wrong server address	Test connection reports the failure plainly	ApiUrlTest, manual
9	Type a server address with no scheme	Accepted; http:// is added	ApiUrlTest
10	Save a profile with a blank first name	Rejected on both client and server	Smoke test
11	Save a three-digit postal code	Rejected on both client and server	ValidatorsTest, smoke test
12	Change password with the wrong current one	Refused (401)	Smoke test
13	Open an admin route as a customer	Refused by the server (403)	Smoke test
14	Sign in as staff on the phone	Told the app is for customer accounts	Manual
15	Switch to dark mode on every screen	No unreadable text; setting survives a restart	Manual, ThemeModeTest
16	Rotate the device on each screen	No crash; the screen reloads its data	Manual

Table 17: The manual verification log

EVIDENCE TO INSERT

Run each case on your own build and record the actual outcome in a fourth column. Add any case specific to changes you made after this document was written.

Case 15 and case 16 are the two most worth filming for the video: they are the ones a marker cannot verify from a screenshot.

11. Help system and user documentation

The module requires user support in two forms, and both are in the repository.

Deliverable	Where	What is in it
In-app help	Account → Help	Eight task-based topics, plus a support panel and a diagnostics panel
User guide	docs/user-guide.md	Installing, connecting, buying, tracking, settings, troubleshooting, staff console, and instructions for the marker
README	README.md	Setup, demo accounts, SSO configuration, testing, project structure, release notes

Table 18: The three pieces of user documentation

The help topics are written as tasks a customer is trying to finish, such as "Track an order", "Buy something" or "Switch to dark mode", rather than as a tour of the screens. Someone who is stuck does not know what we called the screen they need, so indexing the help by our screen names would only help a person who did not need help.

The Help screen ends with a panel showing the app version, the server address it is currently using and who is signed in. That is there for support rather than for decoration: almost every report of "it isn't working" turns out to be the wrong server address, and this saves walking somebody through three screens to read it out.

The in-app help and docs/user-guide.md carry the same content deliberately, and both files say so, so that changing one without the other is an obvious omission rather than an invisible one.

EVIDENCE TO INSERT

A screenshot of the in-app Help screen, including the diagnostics panel at the bottom.

Confirm that docs/user-guide.md and the README are committed and readable on GitHub.

12. Demonstration video

The demonstration video is recorded on a physical Android handset, not an emulator, and carries a voice-over explaining what is being shown. It is uploaded to YouTube as unlisted, and the link appears in the README as well as here.

The module sets three conditions on this video, and all three should be visibly satisfied: a real phone, a voice-over, and evidence of the data held in the online authentication service, in the API and in the database. The last of these is the one most often missed: it means showing the stored records, not only the screens that were built from them.

12.1 What the walk-through covers

#	Segment	What to show
1	The phone	The handset itself, so it is clear this is not an emulator
2	Sign in	Email and password, then sign out and sign in again with Google
3	Authentication service	The stored identity in the Google Cloud console
4	Browse and search	The Shop grid, brand and category filters, a keyword search
5	Product detail	Size, condition grade, description, reviews
6	Buy	Add to cart, checkout, order confirmation with the reference
7	Track	The Orders tab, then the order detail with the tracker
8	Settings	Change a profile detail, then switch to dark mode live
9	Bad input	A wrong password, an over-order, a wrong server address; none of them crash
10	Help	The Help screen and the diagnostics panel
11	The API	The server log showing the requests the phone just made
12	The database	The rows behind what was just done, including the hashed passwords
13	The website	The same order in the console; set it to Shipped and see it change on the phone
14	The pipeline	A green GitHub Actions run with the test counts

Table 19: Suggested running order for the demonstration video

Segment 13 is the one worth rehearsing. Changing an order's status on the website and then watching it change on the phone is the single clearest demonstration that both clients share one database, and it takes about twenty seconds.

EVIDENCE TO INSERT

Paste the unlisted YouTube link here, and into the README.

Check before submitting: recorded on a real phone; voice-over throughout; shows data in the authentication service, the API and the database.

13. Release notes

Version 1.1.0 (build 2) is the Task 2 prototype. The same notes are kept in the README, which is where the module asks for them.

Area	What changed since the Task 1 baseline
Single sign-on	Added end to end. Credential Manager on the phone, POST /api/auth/sso on the server, with the Google ID token verified for issuer, audience, expiry and a verified email address before any session is issued. Neither the app nor the repository holds the client id.
Account security	The Customer table now records how an account authenticates, with a constraint enforcing that a password account has a hash and an SSO account has a provider subject. Signing in with Google on an address that already has a password account is refused rather than linked. Customers can change their password from the app.
Settings	A new settings screen: profile, password, appearance, notifications, connection and reset. Dark theme, applied before the first screen is drawn. The server address moved here and gained a Test connection button.

Area	What changed since the Task 1 baseline
Help and documentation	A new in-app Help screen with eight task-based topics and a diagnostics panel, and a new written user guide at docs/user-guide.md.
Testing	49 Kotlin unit tests added. Before this the Android CI job ran against an empty source set and passed without testing anything. 9 API unit tests added. The end-to-end suite grew from 51 checks to 70.
Release pipeline	A tag-triggered workflow producing a signed APK and App Bundle, with the keystore held as a repository secret and deleted from the runner afterwards.
Database	Every table now holds at least ten rows and the seed script fails rather than under-filling one. Twelve customers, ten staff accounts, twelve orders across all four statuses, twelve open carts, fifteen reviews. Four categories split out: Polos, Shirts, Hoodies, Jackets.
Logging	Per-request server logging of method, path, status, duration and who was signed in, with failures sent to stderr so they can be filtered out of a CI log.
Validation	Input rules collected into one tested place matching the server's own, so a customer is no longer told a password is fine and then rejected by the API.

Table 20: Release notes for version 1.1.0 (build 2)

Of these, the four that a marker should look at first are single sign-on, the settings screen, the help system and the Kotlin unit tests, because each of them is a rubric line that Task 1 did not address at all.

14. Preparation for Google Play publication

The application is prepared for publication. The release workflow produces a signed APK and an Android App Bundle from a keystore that is never committed, which is the part of publication that has to be got right first: the signing key cannot be rotated once an app is live, so an app signed with a key that leaked, or lost, cannot be updated at all.

Requirement	State
Signed release artefact	Produced by .github/workflows/release.yml on a v* tag
Keystore kept out of the repository	Held base64 in a repository secret; restored to the runner and deleted afterwards
Application id	com.codecuture.ithrift
Version	1.1.0 (versionCode 2)
Target API level	35
App Bundle (.aab) for the Play Console	Produced alongside the APK
Store listing text and graphics	Listing text drafted in Section 14.1; feature graphic and phone screenshots still to capture
Play Console upload	TO BE COMPLETED

Table 21: Publication readiness

EVIDENCE TO INSERT

A screenshot of the signed APK or App Bundle produced by the release workflow.

A screenshot of the app in the Google Play Console, uploaded and, if applicable, published.

Play Store screenshots at the sizes the Console requires.

14.1 Draft store listing

App name (30 characters max): iTHRIFT: Pre-loved Fashion

Short description (80 characters max): Buy condition-graded, pre-loved branded clothing in South Africa.

Full description: iTHRIFT sells pre-loved branded clothing, footwear and accessories that our team has checked and graded before listing. Every piece carries an honest condition grade, Excellent, Very Good or Good, so you know what you are buying before it arrives. Browse by category and brand, search for the piece you want, and check out in a few taps. Track your order from Processing to Shipped to Delivered, with the courier reference in the app. Sign in with your email or with Google, switch between light and dark themes, and manage your delivery details in Settings. Prices are in rand and delivery is within South Africa.

Category and details: Shopping. Content rating: Everyone. Contact: the group's student email addresses.

15. Use of AI tools

The group used Claude, a large language model made by Anthropic, through the Claude app during Task 2 (Anthropic, 2026). It was used as a pair programmer and an editor, not as a substitute for understanding the code, and this section sets out where.

Code. Claude produced first drafts of the new Settings and Help screens, the Google single sign-on flow on both the phone and the server, the Validators object, and most of the Kotlin and API unit tests. It also helped diagnose Gradle and Android Studio set-up errors on Windows. Each draft was read, run and changed where it did not fit: for example, the rule that a Google sign-in on an address that already has a password account is refused rather than linked was a decision the group made after discussing the security trade-off.

Documentation. Claude drafted the structure and much of the wording of this document, the README and the user guide from the group's notes and the code in the repository. The group edited the drafts for accuracy against what was actually built.

Evidence and checking. In the final session Claude ran the API unit tests and the end-to-end smoke test in a sandbox, captured the website and database screenshots in Figures 1 to 12, validated the HTML of every page and made the Task 1 feedback fixes listed in Section 1.5. The screenshots are of the real system running from this repository; none is a mock-up.

How the output was checked. Nothing was accepted on trust. Every rule described in this document is covered by an automated test that runs in GitHub Actions, and a suggestion that could not be explained by the member committing it was not committed. Where Claude and the tests disagreed, the tests won.

What it was not used for. The phone recordings, the demonstration video and voice-over, the Azure Boards history and the peer and self evaluations are the group's own work.

CHECK BEFORE SUBMITTING

This account was drafted for the group. Read it against what each of you actually did, correct anything that is not accurate, and add anything that is missing. It must stay under 500 words.

16. Requirement checklist

Every requirement in Section 9.2 of the module manual, and every criterion on the Task 2 marking rubric, mapped to where it is met.

16.1 Against the marking rubric

Rubric criterion	Marks	Where it is met
The app runs on a mobile device	10	§4, §12: builds to app-debug.apk (about 8 MB); demonstrated on a handset in the video
Feature: sign in	10	§5.1, §5.3: email and password, plus Google single sign-on
Feature: settings menu	10	§4.5: profile, password, appearance, notifications, connection, reset
Creation and use of the API	10	§7: the shared REST API, used by both clients
Integration of the API in the app	10	§4.1, §4.4: Retrofit client, typed results, every screen
Database structure and table design	15	§6: 11 tables in 3NF, constraints, ≥10 rows in every table
User interface	10	§4.3: five tabs, two-column grid, light and dark themes
User documentation	10	§11: in-app Help, docs/user-guide.md, README
GitHub, README and automated testing	10	§3, §9: three workflows, 128 automated checks, no ZIP files
Demonstration video	10	§12: real handset, voice-over, shows the stored data

Table 22: The Task 2 marking rubric, mapped to this document

16.2 Against the module manual, Section 9.2

Requirement	Where it is met	Done
Mobile app developed on GitHub	§4, repository link in §1.3	Yes
Website updated	§8	Yes
Full DevOps lifecycle	§3	Yes
Azure DevOps used throughout implementation	§3.1	Insert board evidence
Runs on a phone, not an emulator	§12	Record the video
GUI, input controls and program modules coded	§4.2 to §4.4	Yes
Database tables designed in a DBMS	§6.1 to §6.3	Yes
At least ten records per table	§6.4	Yes
System verified and tested	§9, §10	Yes
Help system and user documentation	§11	Yes
Register and log in	§5.1	Yes
Password encrypted	§5.2	Yes
Single sign-on	§5.3	Yes (needs a client id to demonstrate)
User can change their settings	§4.5	Yes
UI handles invalid input without crashing	§4.4, §10	Yes
Automated testing through GitHub Actions	§3.3, §9	Yes

Requirement	Where it is met	Done
Video: real phone, voice-over, shows auth/API/DB data	§12	Record the video
Cover page and table of contents	This document	Yes
GitHub links in the document; no ZIP files	§1.3	Yes
Release notes since the prototype	§13	Yes
Prepared for Google Play publication	§14	Partly; listing drafted, upload to do
Write-up on the use of AI tools	§15	Drafted; check it

Table 23: Section 9.2 requirements, mapped to this document

The items not yet ticked need something only the group can supply: the Azure Boards link and a screenshot of the board, the demonstration video recorded on a phone, the Play Console upload, and a final check of the account of AI use. Everything else is in the repository and can be verified by running it.

References

Anthropic. 2026. Claude [Large language model]. Available at: <https://claude.ai> [Accessed 25 September 2026].

Google. 2025. Guide to app architecture. [Online]. Available at: <https://developer.android.com/topic/architecture> [Accessed 6 September 2026].

Google. 2025. Sign in your user with Credential Manager. [Online]. Available at: <https://developer.android.com/identity/sign-in/credential-manager-siwg> [Accessed 6 September 2026].

Google. 2025. Authenticate with a backend server. [Online]. Available at: <https://developers.google.com/identity/gsi/web/guides/verify-google-id-token> [Accessed 6 September 2026].

Google. 2025. Material Design 3: Navigation bar. [Online]. Available at: <https://m3.material.io/components/navigation-bar/overview> [Accessed 6 September 2026].

Google. 2025. Sign your app. [Online]. Available at: <https://developer.android.com/studio/publish/app-signing> [Accessed 6 September 2026].

GitHub. 2025. GitHub Actions documentation. [Online]. Available at: <https://docs.github.com/en/actions> [Accessed 6 September 2026].

GitHub Marketplace. 2025. Automated build Android app with GitHub Action. [Online]. Available at: <https://github.com/marketplace/actions/automated-build-android-app-with-github-action> [Accessed 3 November 2025].

Microsoft. 2025. Azure Boards documentation. [Online]. Available at: <https://learn.microsoft.com/en-us/azure/devops/boards> [Accessed 6 September 2026].

Microsoft. 2025. What is DevOps? [Online]. Available at: <https://learn.microsoft.com/en-us/devops/what-is-devops> [Accessed 6 September 2026].

Node.js. 2025. Crypto: scrypt. [Online]. Available at: <https://nodejs.org/api/crypto.html> [Accessed 6 September 2026].

OWASP. 2021. Password storage cheat sheet. [Online]. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html [Accessed 6 September 2026].

The Independent Institute of Education. 2026. Module manual XISD6329: Work Integrated Learning 3B.
Johannesburg: The Independent Institute of Education.